Caribbean Voices ASR

Zindi Hackathon - Solution Documentation

Team: Di Transcribers | Author: Douglas Byfield and Tia Salmon

1. Overview and Objectives

This solution addresses the challenge of Automatic Speech Recognition (ASR) for Caribbean-accented English, which is underrepresented in standard ASR training data.

Purpose: Fine-tune OpenAI's Whisper Large-v3 model using LoRA (Low-Rank Adaptation) to improve transcription accuracy for Caribbean English speech.

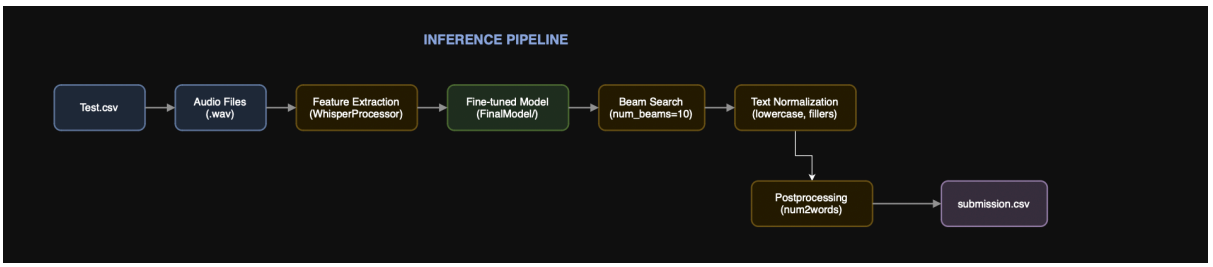
Problem Addressed: Standard ASR models perform poorly on Caribbean accents due to limited training data representation. This solution adapts the model to recognize Caribbean speech patterns, proper nouns, and vocabulary.

Expected Outcome: Reduced Word Error Rate (WER) on Caribbean English audio compared to baseline Whisper, enabling more accurate transcription of Caribbean broadcast content.

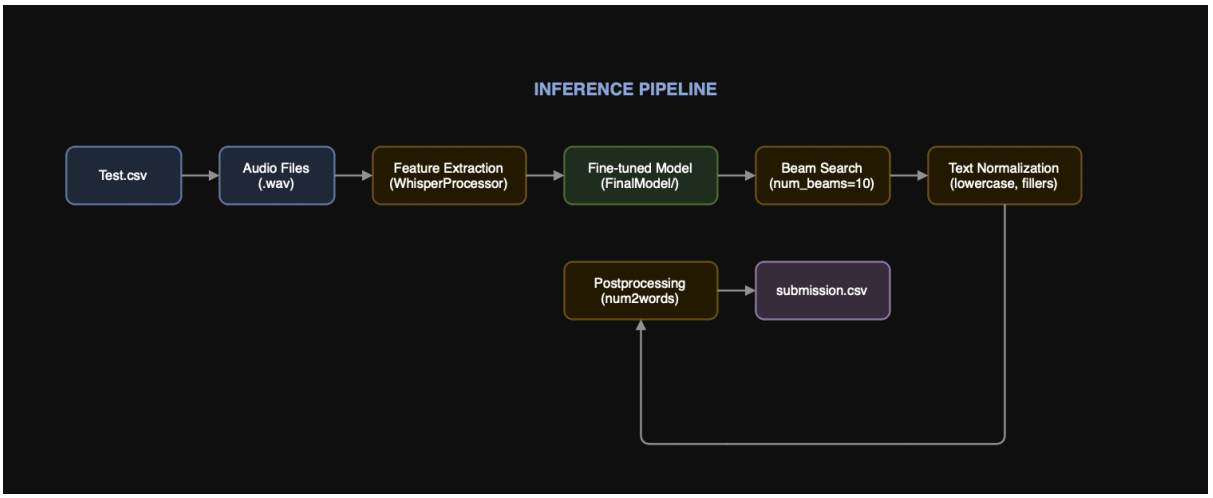
2. Architecture Diagram

Data Flow Overview:

TRAINING PIPELINE:



INFERENCE PIPELINE:



3. ETL Process

Extract

Aspect	Details
Data Sources	Zindi-provided Train.csv, Test.csv, Audio.zip
Data Format	Audio: WAV (mono, 16kHz); Labels: CSV
Volume	~19,856 training samples, ~8,510 test samples
Extraction Method	Direct file loading with librosa library

Transform

Step	Description
Audio Resampling	Convert all audio to 16kHz (Whisper requirement)
Feature Extraction	Log-mel spectrogram via WhisperProcessor
Label Tokenization	Text tokenized using Whisper tokenizer
Train/Val Split	90/10 split with random_state=42

Load

Aspect	Details
Storage	HuggingFace Dataset objects in memory
Batching	DataLoader with batch_size=8
Collation	Custom DataCollatorSpeechSeq2SeqWithPadding

4. Data Modeling

Model Architecture

Component	Details
Base Model	openai/whisper-large-v3 (1.55B parameters)
Fine-tuning Method	LoRA (Low-Rank Adaptation)
Trainable Params	7.8M (0.5% of total)
Frozen Params	1.54B (99.5% of total)

LoRA Configuration

Parameter	Value
Rank (r)	64
Alpha	128
Dropout	0.05
Target Modules	q_proj, k_proj, v_proj, o_proj

Training Hyperparameters

Parameter	Value
Learning Rate	2e-5
Batch Size	8
Gradient Accumulation	2 (effective batch = 16)
Max Steps	3,500
Warmup Steps	500
Precision	FP16

Model Validation

Validation performed every 500 steps using held-out validation set (10% of training data). Best model selected based on lowest validation loss. Final model evaluated using Word Error Rate (WER) metric.

5. Inference

Deployment

Aspect	Details
Infrastructure	Single GPU (8GB+ VRAM required)
Model Loading	Base model + LoRA adapter merged
Precision	FP16 for memory efficiency

Generation Parameters

Parameter	Value
Beam Width	10
Temperature	0.0 (deterministic)
Repetition Penalty	1.2
Max New Tokens	444

Post-Processing

Text normalization applied: lowercase conversion, filler word removal (uh, um, er, ah, hmm), duplicate word removal, punctuation standardization, whitespace normalization.

Model Updates

LoRA adapters allow easy weight updates without full retraining. Checkpoints retained for version rollback if needed.

6. Run Time

Section	Time
Training (A100 GPU)	3-4 hours
Training (T4 GPU)	8-10 hours
Inference (A100)	30-45 minutes
Inference (T4)	1.5-2 hours

7. Performance Metrics

Competition Scores

Metric	Score
Public Leaderboard (WER)	0.034076303
Private Leaderboard (WER)	0.034229624

Training Metrics

Metric	Value
Final Training Loss	~0.05-0.08
Final Validation Loss	~0.06-0.10

Evaluation Metric: Word Error Rate (WER) - measures edit distance between predicted and ground truth transcriptions. Lower is better.

8. Error Handling and Logging

Training

Checkpoints saved every 500 steps for recovery. Best model saved based on validation loss. Training logs printed every 100 steps showing loss progression.

Inference

Try/except block around each audio file processing. Failed transcriptions default to empty string to prevent pipeline failure. Progress bar (tqdm) shows processing status. Error messages printed with audio ID for debugging.

9. Maintenance and Monitoring

Monitoring

Training loss monitored for convergence. Validation loss checked for overfitting. GPU memory usage tracked via nvidia-smi.

Scaling

Batch inference supported for higher throughput. Model can be quantized (INT8) for edge deployment. Multi-GPU training supported but not required.

Lifecycle Management

LoRA adapters (~400MB) easily versioned and stored. Base Whisper model remains unchanged. New adapters can be trained on additional data without full retraining.

File Structure

submission/

```
├── manfile.ipynb                # Jupyter notebook
├── requirements.txt           # Dependencies
├── README.md                  # Documentation for notebook
└──FinalModel/  # Model weights of best model
    ├── adapter_config.json
    ├── adapter_model.safetensors
    └── training_args.bin
```

How to Run

Install dependencies

pip install -r requirements.txt

Run

Open the notebook and run all cells

Hardware Used

Component	Specification
GPU	A100
CUDA Version	12.1
Python	3.10
PyTorch	2.3.1